Aumen, Christopher

CSCI-499 Senior Project

April 20, 2018

in partial fulfillment of the requirements for the degree of bachelor of science

Senior Project Design Document

Section 1 – Project Description

1.1 Project Title Information

Charleston Southern University Competitive Programming Framework

"Arbiter"

Created by: Christopher A. Aumen, Student 163119

Advised by: Dr. Paul E. West, Assistant Professor of Computer Science

1.2 Project Description

This project will implement a framework for conducting a programming competition at a local level, specifically on the Charleston Southern University Campus. The goal is to provide a system that the university's competitive programming team can use to hone their skills for the competitions they compete in.

1.3 Goals

Date Achieved:	Goal Description:	Goal Date:	Goal Met:
18-APR-2017	Complete Design Documentation	18-APR-2017	✓
29-NOV-2017	Implement Modules	29-NOV-2017	✓
15-DEC-2017	Integration Test	15-DEC-2017	✓
27-FEB-2018	System Test	27-FEB-2018	✓
20-Apr-2018	Defense Presentation	20-APR-2018	✓

Senior Project Design Document

Contents

Senior Project Design Document

Section 2 – Project Outline

2.1 Student Name: Christopher A. Aumen

2.2 Advisor Name: Dr. Paul E. West

2.3 Expected Date of Graduation: May 2018

2.4 Description of Project

This project will implement a framework for conducting a programming competition. It will provide an intranet portal that hosts general information, a scoreboard, a registration form, and problem sets. The project will implement a medium for participants to submit source code for evaluation (GIT). Finally, it will utilize server side scripts to pull submitted source code, update the scoreboard, and input test cases into the submitted code and evaluate output. This project will NOT provide automated problem sets, each competition will require a proctor that provides problem descriptions and test cases. The proctor will also be required to update pieces of the intranet portal, database, and a configuration file for the server scripts to work.

2.5 Implementation Language(s)

Intranet Portal: HTML, PHP via Apache Web Server

Database: SQL via MySQL Server

Scripts: PHP & BASH

2.6 Hardware/Software Equipment Required

Server machine that provides web server, SQL server, and can be accessed at intended competition location. Server must have internet connection to connect to GITHub.

Client Machine for proctors that can ssh to server.

Client Machines for each team competing (may be provided by competitor). Must have GIT and up-to-date modern web browser (IE, Firefox, Chrome, etc.).

Senior Project Design Document

Section 2 – Project Outline

2.7 Motivation and Problem Statement

This project was conceived to improve Charleston Southern's ability to train a competitive programming team. The capability to introduce students to a competitive environment on-site would allow instructors to better understand where improvements can be made before other competitions such as the International Collegiate Programming Contest (ICPC).

The project will provide experience creating a web site, database, and constructing scripts. It will also require a deeper understanding of GIT and GITHub and networking protocols.

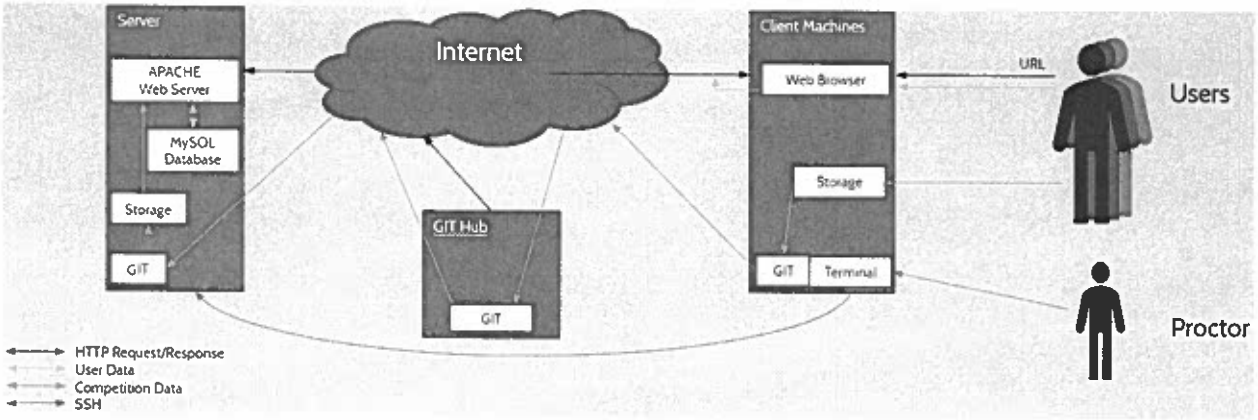
Senior Project Design Document

Section 3 – Hardware Design

3.1 Overview

This project will require at least one server machine capable of hosting a SQL database, a web server, and can connect to GIT Hub. Additionally a minimum of two client machines that have a standard web browser (Firefox, Chrome, IE), access to GIT Hub, and can compile Java and C++.

3.2 Diagram



Senior Project Design Document

Section 4 – GIT Setup

4.1 Overview

A primary administration account will be registered with GITHub under the user name CSUniv_Judge. This account will be used by proctors for setting up the competition problems. The proctor will create a new repository linked to the server machine and grant access to each competitor before the competition. At the commencement of the competition, the problems will be pushed to GITHub and users will be able to pull the problem sets. When a user pushes to their repository, the server will pull the update onto the server machine for testing.

4.2 Configuration

Username: ArbiterCSU

Password: [Default: Prov31:9]

Primary Repository Location: SERVER//~/var/Arbiter/

4.3 Triggers

User submits file: Webhook sends request to webserver for PHP script. PHP script updates local repo, validates source code, compiles code, runs against test cases, updates database with results.

Senior Project Design Document

Section 5 – Database Design [SQL]

5.1 Overview

The database will be used to store and track competitors as they compete. It will store the user's student ID (primary key), git account, first name, last name, problem status (not-attempted, submitted, rejected, accepted), number of problem submissions, and time the problem was accepted.

5.2 Table View (Example)

id	gitun	fName	lName	email
163119	Hiderplo	Christopher	Aumen	caaumen@csustudent.net
123456	smithJ0	John	Smith	Smithj0@email.com
654321	smithJ1	Jane	Smith	Smithj1@email.com
135790	smithW	Will	Smith	Smithw@email.com

5.3 Tables

User: Stores set of Users with primary key of student ID.

Team: Stores set of Teams with primary key of team ID (sequential numbering).

TeamMembers: Association between User and Team.

Competition: Stores set of Competitions with primary key of comp ID (sequential numbering).

CompetitionQuestions: Association between Competition question ID and master record ID (file storage, not database).

Scoreboard: Association between Team and Competition with status for each problem.

Senior Project Design Document

Section 5 – Database Design

5.4 Interface

Inputs

User: Web-form values updates database via PHP script.

Team: Web-form values updates database via PHP script.

TeamMembers: Web-form values updates database via PHP script.

Competition: Bash script run by proctor.

CompetitionQuestions: Bash script run by proctor.

Scoreboard: Web-form values updates database via PHP script.

Outputs

Scoreboard View: Sent to Intranet Web Portal

Senior Project Design Document

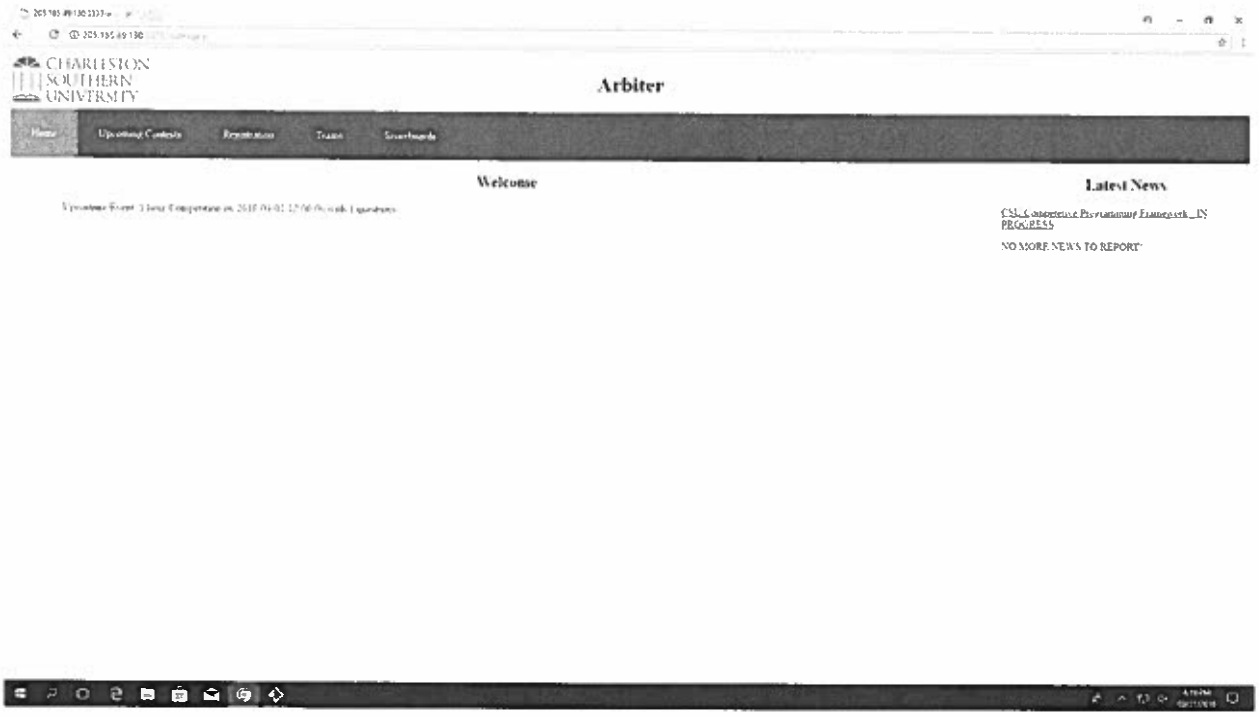
Section 6 – Web Portal Design [HTML/JavaScript]

6.1 Overview

The Intranet portal will be the competitors touchstone for registering and viewing current status of the competition. It will use simple HTML webpages to display contest information, and scripts for getting database information. It will also house a registration form for any upcoming contests that allow the user to apply for the contest.

6.2 Home Page

Displays the next scheduled contest and other information and news as deemed appropriate by CSU Computer Science Department Staff.

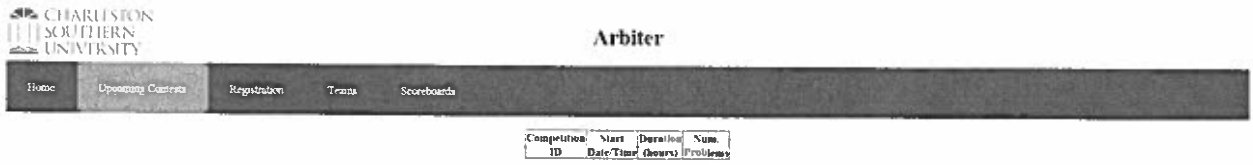


Senior Project Design Document

Section 6 – Web Portal Design [HTML/JavaScript]

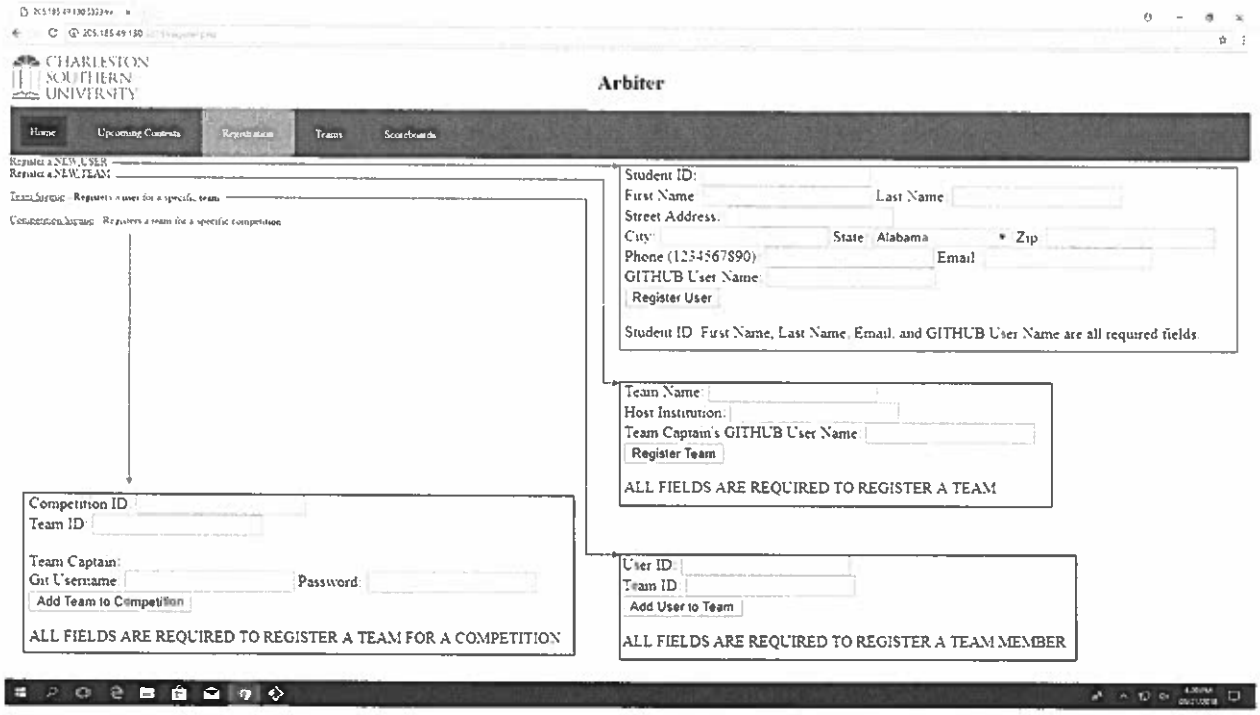
6.3 Upcoming Contests

Lists all scheduled contests in a simple table of Competition ID, Date, Duration, and Problem set size.



6.4 Registration

Form used to add contestants and teams to the database. Process by which users sign-up for a competition.

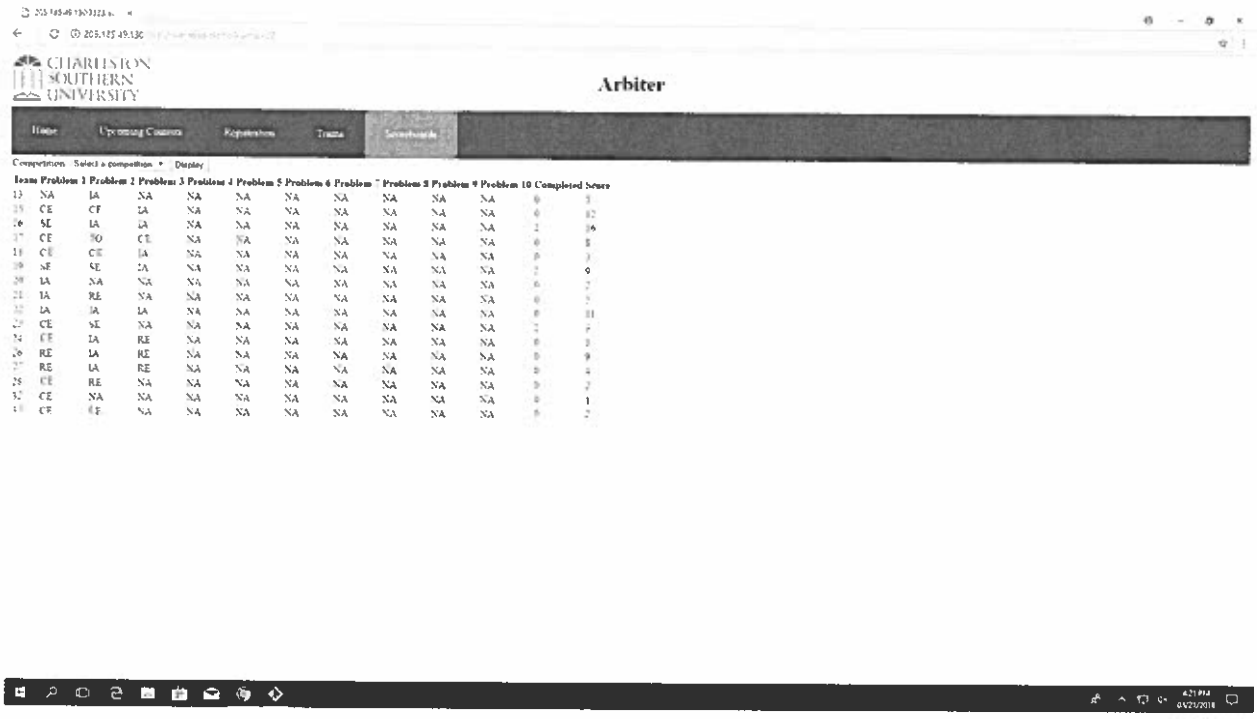


Senior Project Design Document

Section 6 – Web Portal Design [HTML/JavaScript]

6.6 Latest Scoreboard

Displays the scoreboard view of the most recently completed contests database.



Senior Project Design Document

Section 7 – Testing

7.1 Overview

Module testing completed as each module was implemented or modified.

Integration testing included a mock competition created, administered, and taken by project designer.

System test included a live competition created by project sponsor, administered by project creator, and taken by a group of classmates.

Post-system test survey was conducted for feedback.

7.2 Testing Results

Users found the program to be functionally capable, but aesthetically lacking. Additionally security and validation concerns were discovered and will require addressing in future versions

Senior Project Design Document

Section 8 – Future Work

8.1 Summarized List

- Form Validation
- Web-UI HTTPS Compliant
- Create Proctor UI
- Data Encryption
- Interface Aesthetical Enhancements
- Additional Language Support (C#, C, Kotlin, etc.)
- Overhaul Problem Creation Process
- Overhaul User Registration Process
- Add News Feed Functionality
- Add Competition Forums
- Improved Configuration Options

8.2 Cybersecurity

Arbiter is lacking in many areas of security. Forms are insufficiently validated, the web-UI isn't HTTPS compliant, passwords and data are stored unencrypted. For these reasons and others the project author may not realize, a cybersecurity skilled individual is requested to fix these issues and identify and resolve other vulnerabilities.

8.3 User Interface

Many test users pointed out the user interface was lacking in several ways. First of all it isn't 'pretty'. Additionally the registration process was extraneous and could use a lot of optimization. Someone proficient with user interface design could take this program from merely functional to actually appealing. Along with this some new interface features such as designing and implementing the news feed and some forums would be a great upgrade. Finally a ground-up design for an extension of the web-UI to include a proctor portal to eliminate the need to ssh into the server and manually run bash scripts.

Senior Project Design Document

Section 8 – Analysis of Alternatives

8.4 Miscellaneous

The program is in many ways bare-bones. Functionally it works, but it could use a lot of minor enhancements to make it better for training, competing, and user friendly. Currently competitions are set for a single format with limited language options. Adding language support for additional programming languages would increase its usability. With that, adding configuration options for proctors to restrict through whitelist or blacklist which languages a competition will support will allow competitions to be more focused. Additionally the system currently allows teams of any size, both to be created and to compete. Allowing proctors to designate team size restrictions for a competition and enforcing it through the registration process would eliminate some potential foul play cases. Additionally providing a mechanism for team captains to control their team members (add, remove, etc.) would prevent teams from becoming inflated and unusable. Lastly, allowing competitions to provide more constructive feedback would greatly influence the systems use as a training tool. Currently the only feedback provided is through the scoreboard which basically tells the competitor whether or not the submission passed or failed. Creating a feedback file and pushing it to the user's fork with a more detailed description of when and how the submission failed would enhance the training aspect of the program. Additionally this feature should be a configuration option for proctors to toggle on/off when creating the competition.

Senior Project Design Document

Section 9 – Lessons Learned

9.1 Be Realistic

As a full-time undergraduate student the scope of this project from the very beginning was massive. Over estimating capabilities and underestimating timeframes led to frustration and desired features being cut from the plan.

9.2 Scope Creep

While some features planned from the beginning had to be cut, others had to be added. Some functions required additional components to work properly. Other features were added as the idea appealed to the author. All of this compounded the workload while the deadline remained set in stone.

9.3 Plan for Failure

The original schedule planned each task with little to no leeway. Some tasks that were expected to be quick and painless ended up requiring hours extra work and research. Since the plan didn't include time for this extra work, the plan failed.

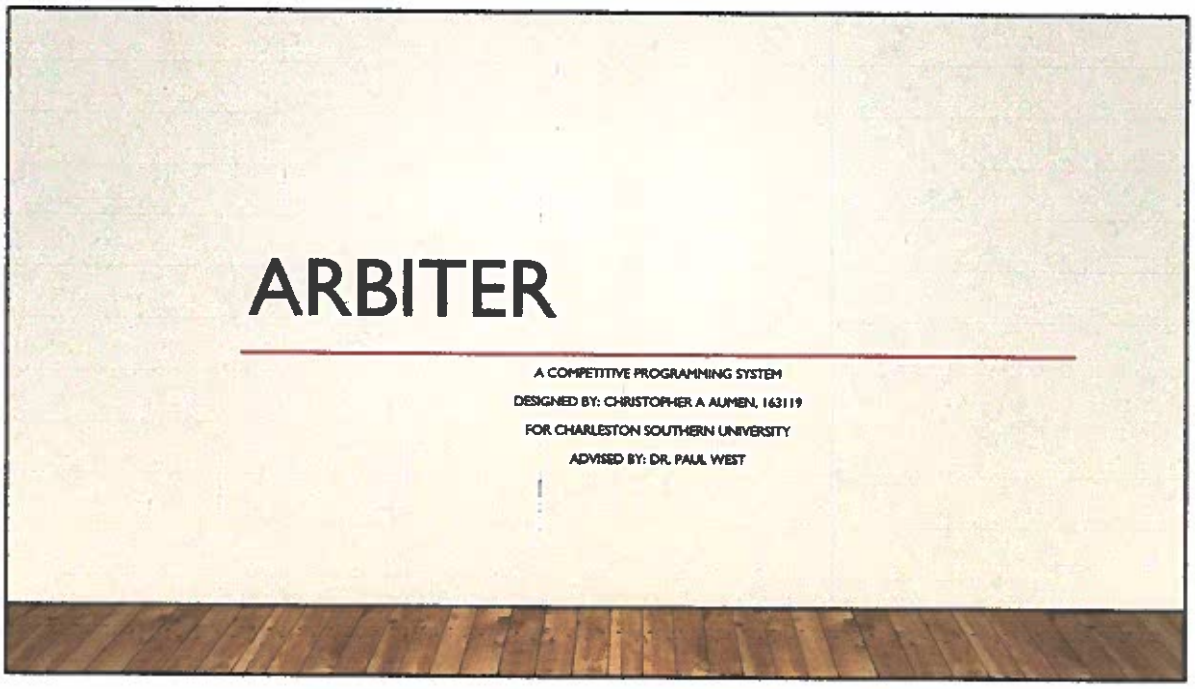
```
<?php
$now = time();
include '/usr/local/mysql/web.php';
$status = "";
$json = file_get_contents('php://input');

$payload = json_decode($json);

function wtlog($mf, $uf, $d) {
      //
      // NEED TO IMPLEMENT A LOGGING FUNCTION, BELOW CODE NOT WORKING
      //


      /*
      if (file_exists($mf)) $mh = fopen($mf, 'a') or die("{$d}\nERROR: {$mf} could
not be openned.");
      else {
            $mh = fopen($mf, 'w') or die("{$d}\nERROR: {$mf} could not be
created.");
            print("\nCreated master log file {$mf} :: {$mh}.\n");
      }
      fwrite($mh, $d);
      fclose($mh);
      if (file_exists($uf)) $uh = fopen($uf, 'a') or die("{$d}\nERROR: {$uf} could
not be openned.");
      else {
            $uh = fopen($uf, 'w') or die("{$d}\nERROR: {$uf} could not be
created.");
            print("\nCreated user log file {$uf} :: {$uh}.\n");
      }
      fwrite($uh, $d);
      fclose($uh);*/
}

if (!empty($payload)) {
    // ************************************************
    // This section sets up configuration items
    // ************************************************
    $STATUSCODE = "SE";
    $repository = $payload->repository->name;
    $username = $payload->repository->owner->login;
    $shun = escapeshellcmd($username);
    $repourl = $payload->repository->html_url;
    $repoloc = "/var/www/gavel/competitionArea/";
    $PROBLEMS = "/var/www/gavel/Problems/";
    $ADDEDPATH = $payload->head_commit->added[0];
    $MODPATH = $payload->head_commit->modified[0];
    $GITPATH = $compTitle = $lQN = $FNAME = $compDate = $compID = $FPATH = $FILEEXT
= $CLASSNAME= 0;
    if ($ADDEDPATH != "") $GITPATH = $ADDEDPATH;
    elseif ($MODPATH != "") $GITPATH = $MODPATH;
    else $status .= "NO FILES CHANGED\n";
    if ($GITPATH != 0) {
      list($compTitle, $lQN, $FNAME) = explode("/", $GITPATH, 3);
      list($compDate, $compID) = explode("_", $compTitle, 2);
      list($CLASSNAME, $FILEEXT) = explode(".", $FNAME, 2);
      $FPATH = "/var/www/gavel/competitionArea/{$username}/arbiter/${GITPATH}";
      //$status .= "GITPATH: {$GITPATH}\ncompTitle: {$compTitle}\nlQN:
```

```
{$lQN}\nFNAME: {$FNAME}\ncompDate: {$compDate}\ncompID: {$compID}\nFPATH:
{$FPATH}\nFILEEXT: {$FILEEXT}\nCLASSNAME: {$CLASSNAME}\n";
    }
    /*if (!file_exists("logs/{$compTitle}/")) {
      system("mkdir logs/{$compTitle}/");
    }
    $masterLogPath = "logs/{$compTitle}/masterLog";
    $userLogPath = "logs/{$compTitle}/{$username}Log";*/

    // ************************************************
    // This section checks if submission is done on time
    // ************************************************
      $dateTimeQuery = "SELECT * FROM Competition WHERE compID = '{$compID}'";
      $compDateTime = "";
      $compDuration = -1;
      if (mysqli_multi_query($conn, $dateTimeQuery)) {
      $dateTimeResult = mysqli_store_result($conn);
      if ($row = mysqli_fetch_assoc($dateTimeResult)) {
              $compDateTime = $row["compDate"];
              $compDuration = $row["timeLimit"];
        }
      } else {
            $status .= "FATAL: Unable to query Competition table of database, check
network configuration.\n";
            wtlog($masterLogPath, $userLogPath, $status);
            die("${status}");
      }
      if ($compDateTime != "" && $compDuration != -1) {
            $sdt = new DateTime($compDateTime);
            $startTime = $sdt->getTimestamp();
            if ($compDuration == 0) {
                  if ($now < $startTime) {
                        $status .= "Submission rejected, competition hasn't started
yet.\n";
                        wtlog($masterLogPath, $userLogPath, $status);
                        die("{$status}");
                  }
            } else {
                  $timeMod = $compDuration * 3600;
                  $endTime = $startTime + $timeMod;
                  //$status .= "start: {$startTime} || now: {$now} || end:
{$endTime}\n";
                  if ($now < $startTime || $now > $endTime) {
                        $status .= "Submission rejected, was not submitted during
competition timeframe.\n";
                        wtlog($masterLogPath, $userLogPath, $status);
                        die("{$status}");
                  }
            }

      } else {
            $status .= "ERROR: Unable to determine competiton start time and
duration.\n";
            wtlog($masterLogPath, $userLogPath, $status);
            die("{$status}");
      }

    // ************************************************
    // This section clones or pulls the users github fork
```

```
    // ************************************************
    if (!file_exists($repoloc . escapeshellcmd($username) . "/arbiter/")) {
        $status = $status . "Cloned user directory.\n";
        system("mkdir " . $repoloc . escapeshellcmd($username) . "/");
        system("chmod -R 777 " . $repoloc . escapeshellcmd($username) . "/");
        sleep(2);
        exec("cd {$repoloc}{$shun}/ && git clone https://{$login}:
{$pass}@github.com/{$username}/arbiter", $output, $retval);
        foreach ($output as $i => $value) $status .= "output[{$i}]:
{$output[$i]}\n";
        $status .= "retval: {$retval}\n";
    } else {
      $status = $status . "Pulled commit.\n";
      system("cd {$repoloc}{$shun}/arbiter/ && git pull");
    }

    // ************************************************
    // This section determines the programming language used
    // ************************************************
    if (substr($FNAME, -3) == "cpp") {
      $status .= "Identified C++ submission, running test cases.\n";
      $of = "/var/www/gavel/competitionArea/{$username}/{$FNAME}.out";
      $compile = "g++ -o {$of} {$FPATH}";
      $run = "{$of}";
      //$status .= "compile: {$compile}\n";
    } elseif (substr($FNAME, -4) == "java") {
      $status .= "Identified JAVA submission, running test cases.";
      $of = "/var/www/gavel/competitionArea/{$username}/";
      $compile = "javac -d {$of} {$FPATH}";
      $run = "java " . substr($FNAME, 0, -5);
    } else {
      $status .= "FATAL: Unknown path extention, please see competition guidelines
for submission procedures.\n";
      wtlog($masterLogPath, $userLogPath, $status);
      die("{$status}");
    }

    // ************************************************
    // This section compiles the submissions, then runs the test cases until all
pass or one fails
    // ************************************************
      $gpn = -1;
      $problemQuery = "SELECT questionID FROM CompetitionQuestions WHERE compID =
'{$compID}' AND question = '{$lQN}'";
      //$status .= "${problemQuery}\n";
      if (mysqli_multi_query($conn, $problemQuery)) {
      $problemResult = mysqli_store_result($conn);
      if ($row = mysqli_fetch_assoc($problemResult)) $gpn = $row["questionID"];
      } else {
            $status .= "FATAL: Unable to query CompetitionQuestions table of
database, check network configuration.\n";
            wtlog($masterLogPath, $userLogPath, $status);
            die("${status}");
      }
      if ($gpn == -1) {
            $status .= "ERROR: Unable to associate local problem number to global
problem number.  Check submission protocol.\n";
            wtlog($masterLogPath, $userLogPath, $status);
            die("${status}");
```

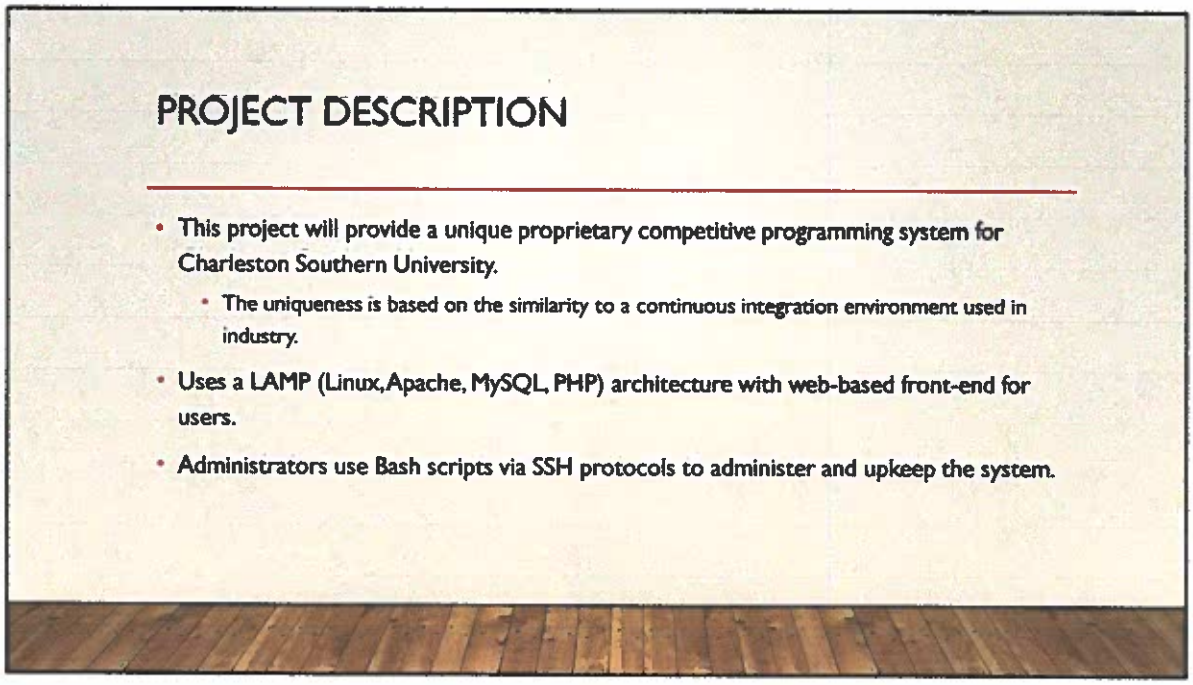
```
      }
    exec($compile, $output3, $retval3);
    if ($retval3 == 0) { //successful compilation
      $testCase = 1;
      while (file_exists("${PROBLEMS}${gpn}/t/${testCase}")) {
        $output4 = "";
        $expected = file_get_contents("${PROBLEMS}${gpn}/t/timeout");
        $timelimit = $expected + 1;
        $cmd = "timeout {$timelimit}s {$run} < '{$PROBLEMS}{$gpn}/t/{$testCase}'";
        $start = time();
        exec($cmd, $output4, $retval4);
        $end = time();
        if ($retval4 == 0) { //successful runtime
          $runtime = $end - $start;
          if ($runtime < $timelimit) {
            $ans = file_get_contents("{$PROBLEMS}{$gpn}/a/{$testCase}");
            $attempt = $output4[0];
          $idx = 1;
          $maxIdx = count($output4);
          while ($idx < $maxIdx) {
                $nextPart = $output4[$idx];
                $attempt .= "\n" . $nextPart;
                $idx += 1;
          }
            if (strcmp(rtrim($ans), rtrim($attempt)) == 0) {
              $status .= "Test case {$testCase} passed.\n";
            } else {
              $status .= "Test case {$testCase} failed.\n";
              $STATUSCODE = "IA";
              break;
            }
          } else {
            $STATUSCODE = "TO";
            $status .= "Test case {$testCase} timed out.\n";
            break;
          }
        } elseif ($retval4 == 124) {
          $STATUSCODE = "TO";
          $status .= "Test case {$testCase} timed out.\n";
          break;
        } else {
          $STATUSCODE = "RE";
          $status .= "Test case {$testCase} encountered a runtime error
{$retval4}.\n";
          break;
        }
        $testCase += 1;
      }
    } else {
      $STATUSCODE = "CE";
      $status .= "Submission encountered compilation error {$retval3}.\nError
Summary:\n";
      //foreach ($output3 as $i => $value) $status .= "output3[{$i}]:
{$output3[$i]}\n";
    }

    // ************************************************
    //This section updates the Scoreboard based on the results
    // ************************************************
```

```
      if ($compID != -1) {
      $teamID = -1;
      $teamQuery = "SELECT Scoreboard.teamID FROM Scoreboard INNER JOIN Team ON
Scoreboard.teamID = Team.teamID WHERE Team.captain = '{$username}' AND
Scoreboard.competitionID = {$compID}";
            if (mysqli_multi_query($conn, $teamQuery)) {
              $teamResult = mysqli_store_result($conn);
              if ($row = mysqli_fetch_assoc($teamResult)) {
                  $teamID = $row["teamID"];
                  if ($teamID != -1) {
                    $updateQuery = "UPDATE Scoreboard SET p{$lQN}Status =
'{$STATUSCODE}', p{$lQN}Score = p{$lQN}Score + 1 WHERE teamID = {$teamID} AND
competitionID = {$compID}";
                    if (mysqli_multi_query($conn, $updateQuery)) {
                        $status .= "Scoreboard updated.\n";
                    } else {
                        $status .= "Scoreboard could not be updated.\n";
                        wtlog($masterLogPath, $userLogPath, $status);
                        die("{$status}");
                    }
                  } else {
                    $status .= "ERROR: Could not identify associated team, seek
assistance from proctor.\n";
                    wtlog($masterLogPath, $userLogPath, $status);
                    die("{$status}");
                  }
              } else {
                  $status .= "ERROR: Could not identify associated team, seek
assistance from proctor.\n";
                  wtlog($masterLogPath, $userLogPath, $status);
                  die("{$status}");
              }
            } else {
              $status .= "FATAL: Could not run teamQuery on database, seek
assistance from proctor.\n";
              wtlog($masterLogPath, $userLogPath, $status);
              die("{$status}");
            }
      } else {
      $status .= "ERROR: Could not identify associated competition, seek assistance
from proctor.\n";

      wtlog($masterLogPath, $userLogPath, $status);
      die("{$status}");
      }
    wtlog($masterLogPath, $userLogPath, $status);
    print($status);
} else {


    wtlog($masterLogPath, $userLogPath, $status);
    die("{$status}");
}

?>
```



For my senior project I wanted to create a system for Charleston Southern's Competitive Programming Team to use for skill enhancement.

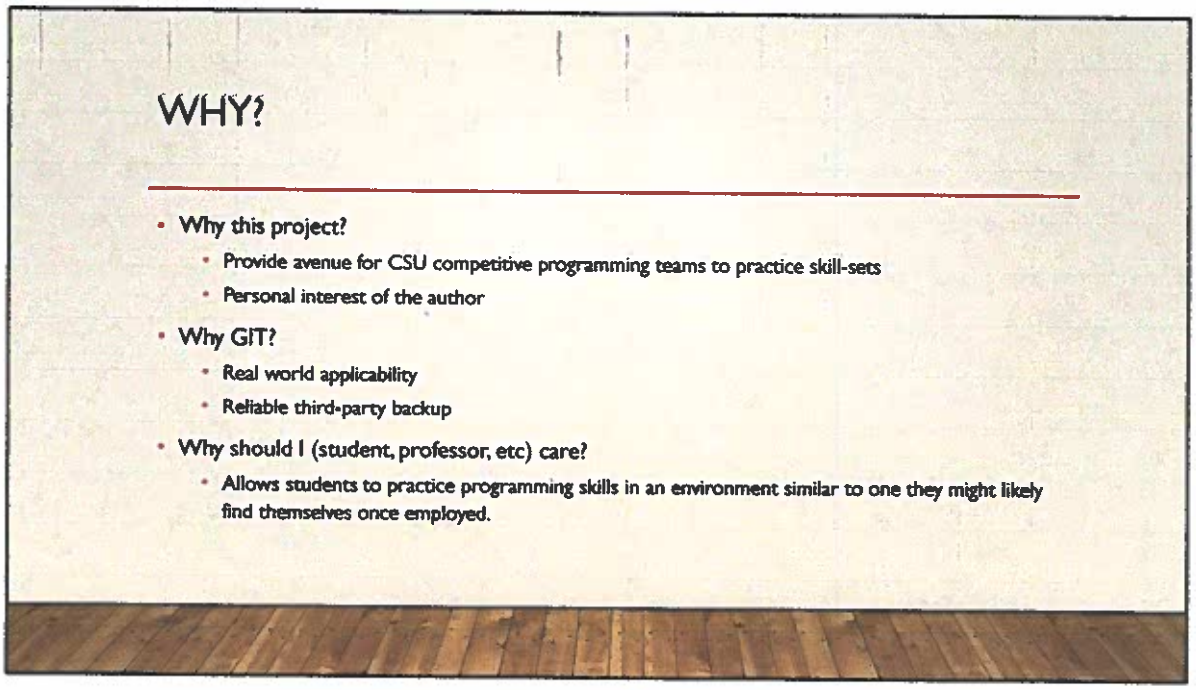
Thus, Arbiter.



Arbiter is a unique take on competitive programming as it works in a continuous integration environment similar to what programmers will find in industry instead of the usual submit-and-forget systems.

--Extra: This means teammates can pick up the code straight from GITHub and provide modifications without having to commandeer their teammates workstation to make corrections.

Using a LAMP architecture, the system provides a web-based front-end for competitors and allows proctors administrative commands through ssh and bash scripts.

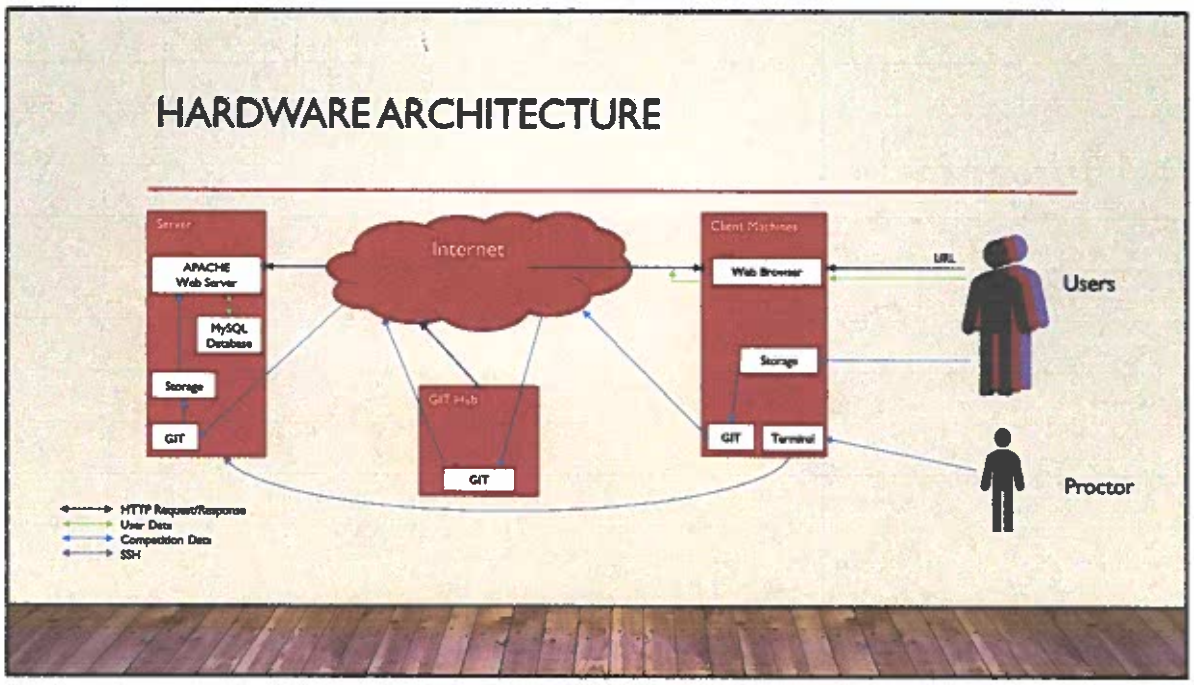


Our programming team was lacking some essential skill-sets during previous competitions that aren't provided through the standard practice system of HackerRank.

In a competition the teams were given a set of problems and a timetable. Successfully placing at higher ranks first and foremost required the ability to solve the problems. Once capability is established however, teams must be able to identify those problems they can solve, and then solve them and as quickly as possible with as few submissions as possible. This fast identification skill was not being developed through HackerRank.

I chose GIT as the medium for submissions because it is a system used in real world applications and developing students experience with GIT will make them more desirable candidates for employment.

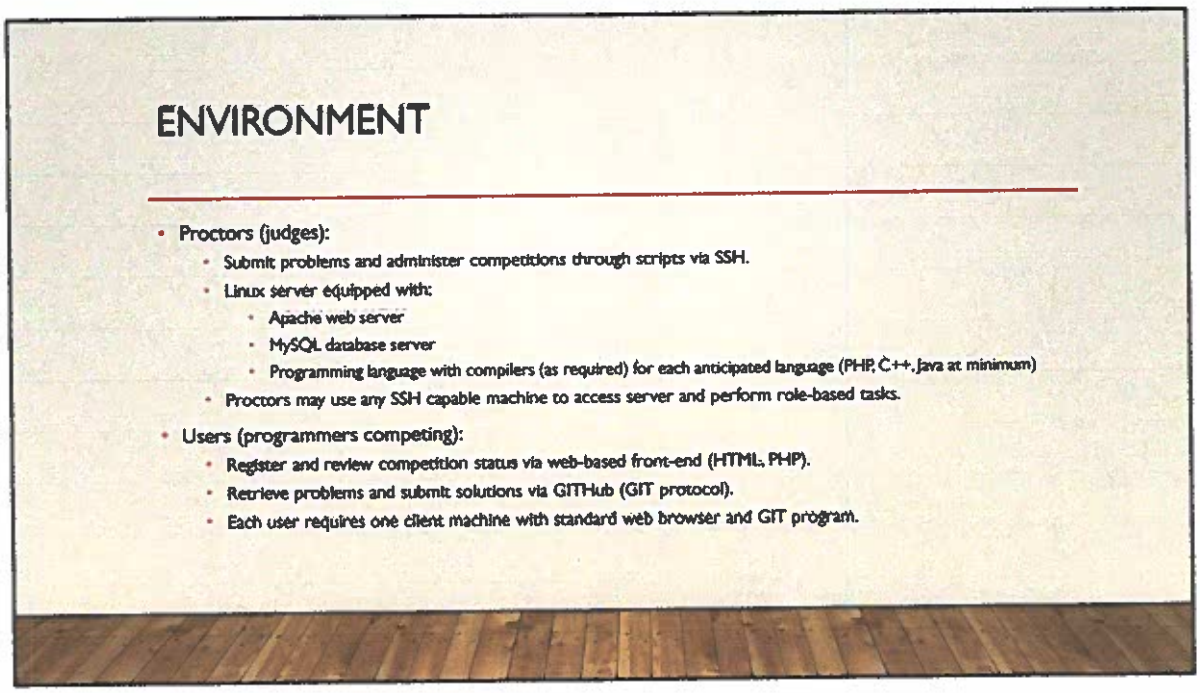
Developing our Competitive Programming team will help the Computer Science Department at Charleston Southern get better recognition as the team improves and brings home more and more top place finishes.



Basic Communication Pathways:

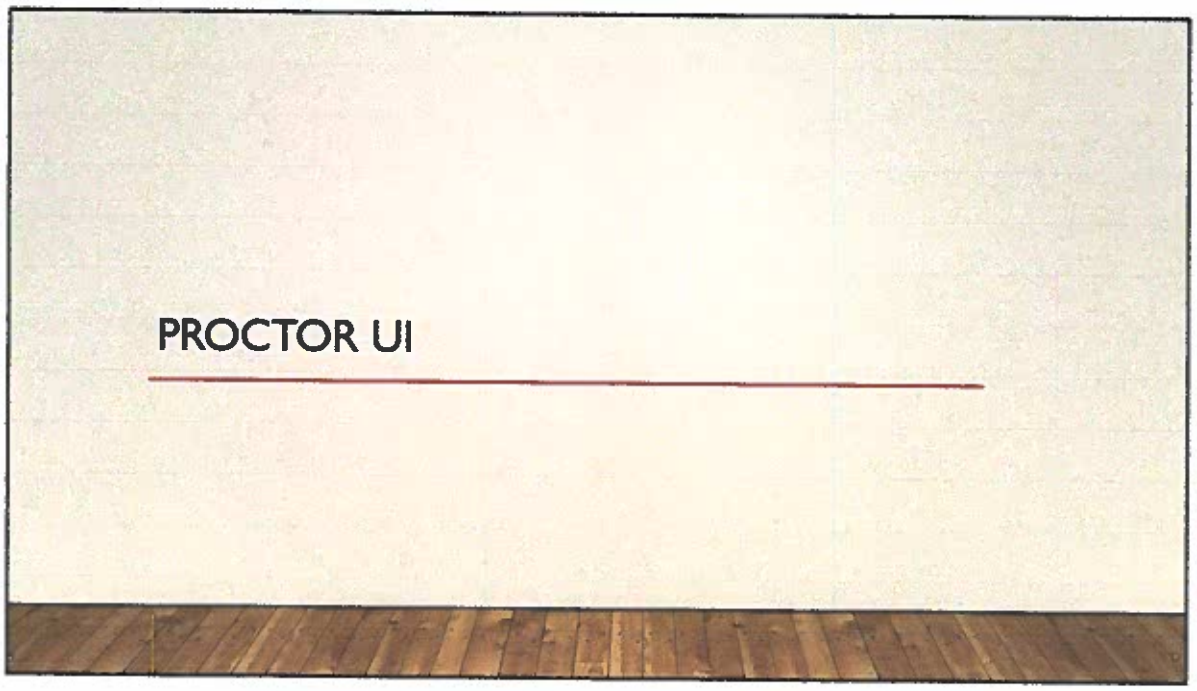
Proctors use a client terminal to ssh into the server. They can then access the bash scripts used to administer a competition.

Users (Competitors) can register and view status on the web portal, create and submit source code on either GITHub's UI or through GIT command line interface.

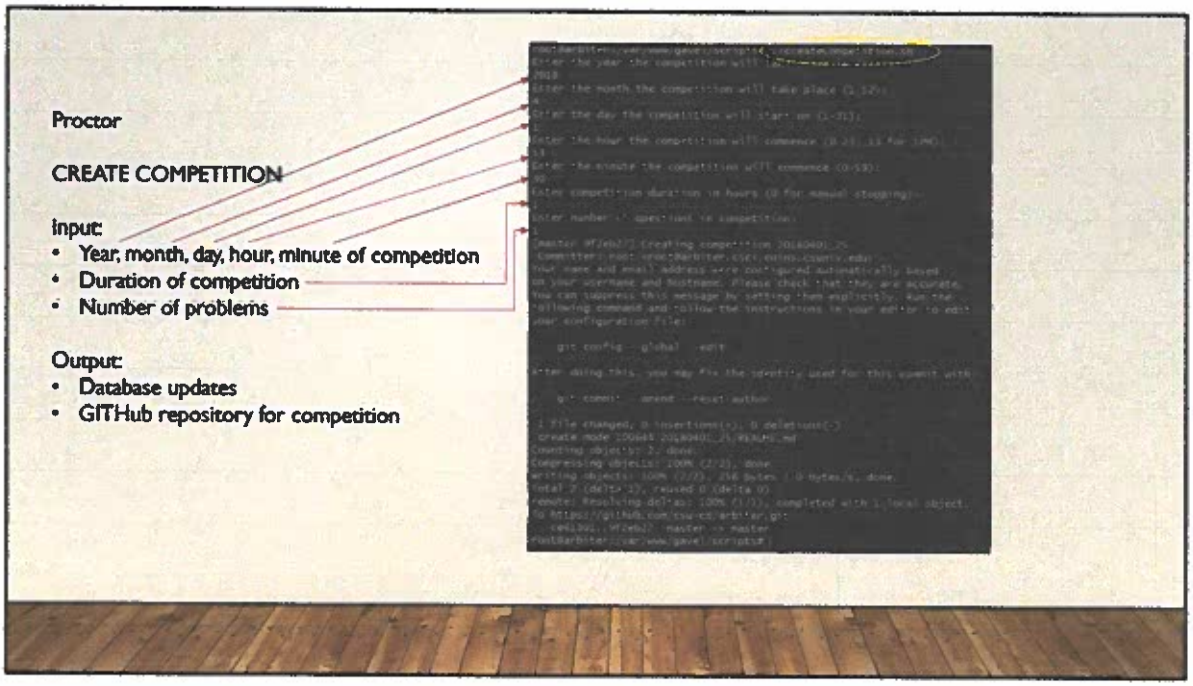


For successful functionality a Linux server with Apache, MySQL, and every desired language compiler installed. Client machines are needed for each competitor with a modern web browser (IE, FF, Chrome, etc) and GIT installed. An additional client machine is needed for the proctor that must be able to ssh into the server.

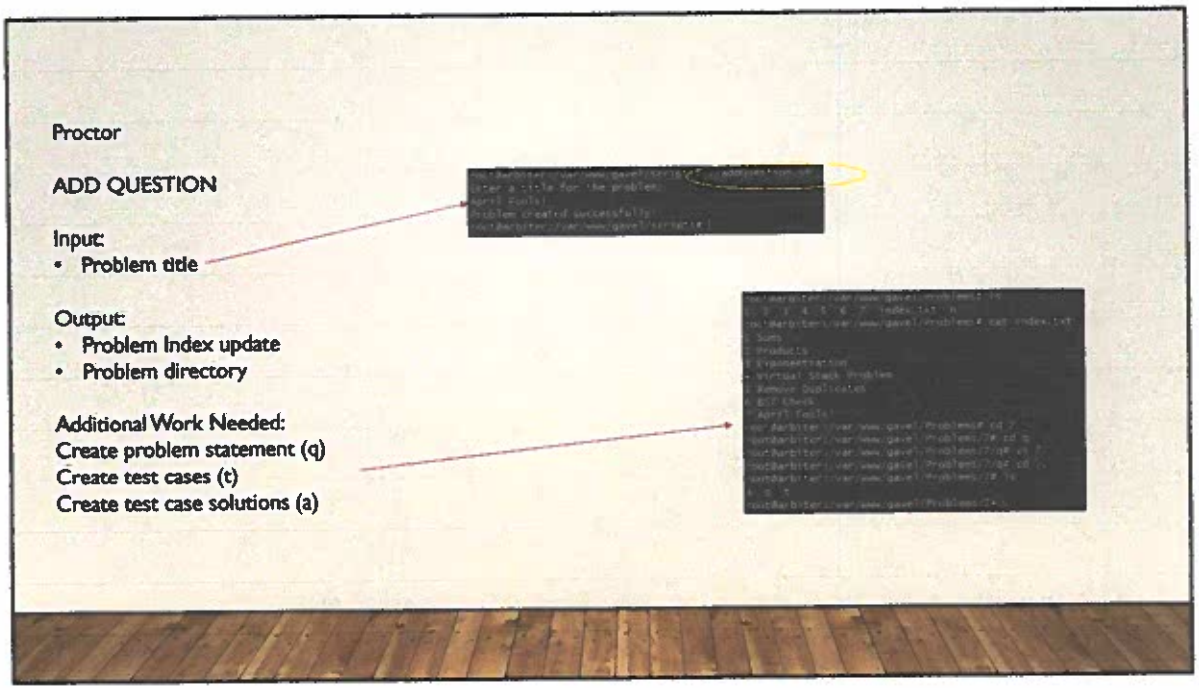
Now to take at look at the system in action



First lets create a competition and get it started



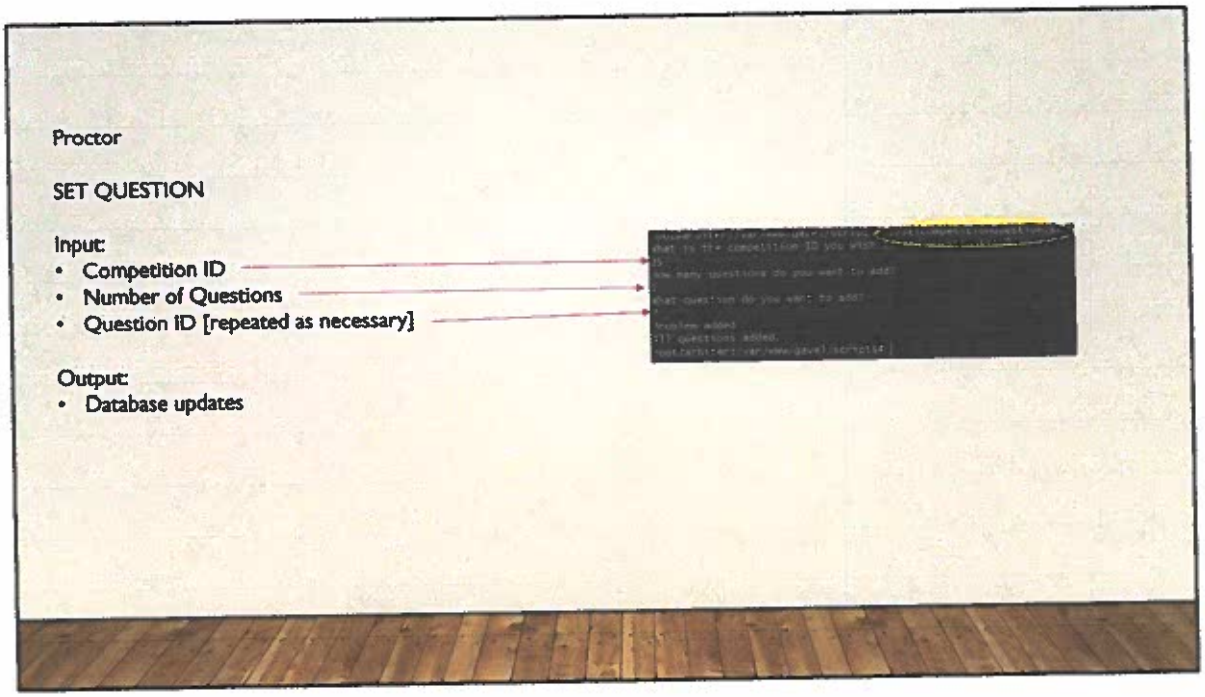
The proctor runs the createCompetition script and enters necessary inputs. The database and GITHub is updated to include the new competition parameters.



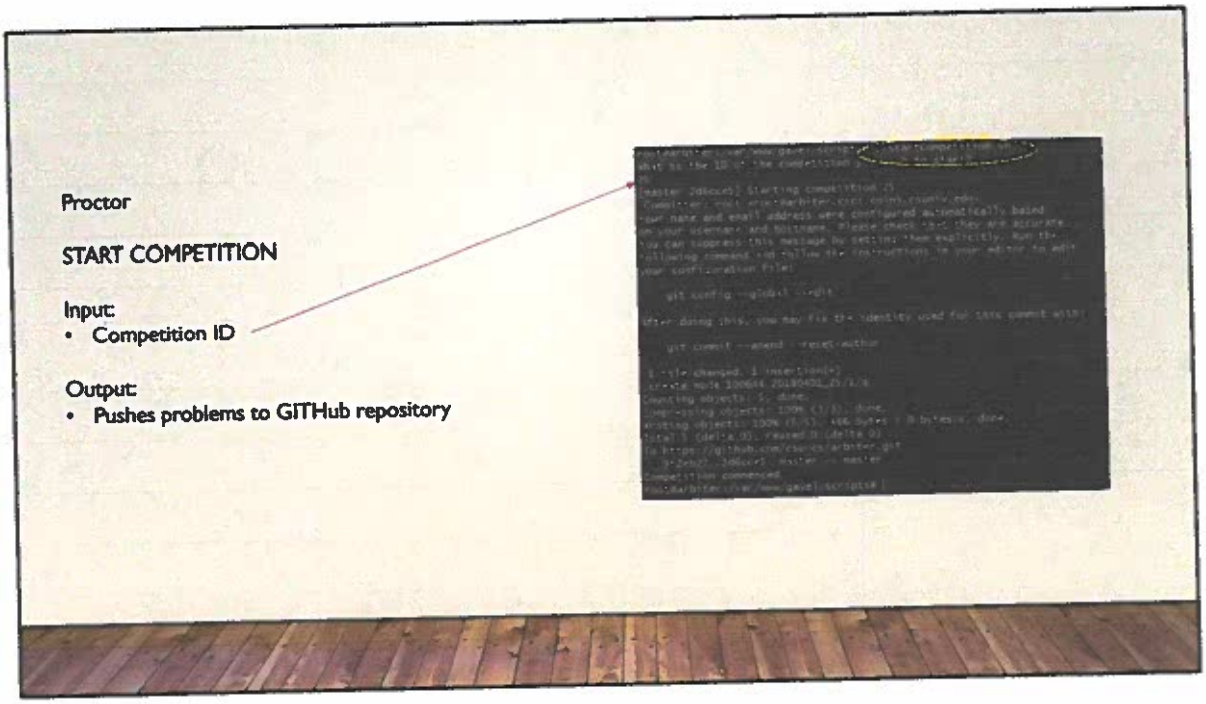
Next problem sets must be established if not already done.

The proctor runs the addQuestion script and enters the title. The directory structure and problem index are updated to accommodate the new problem.

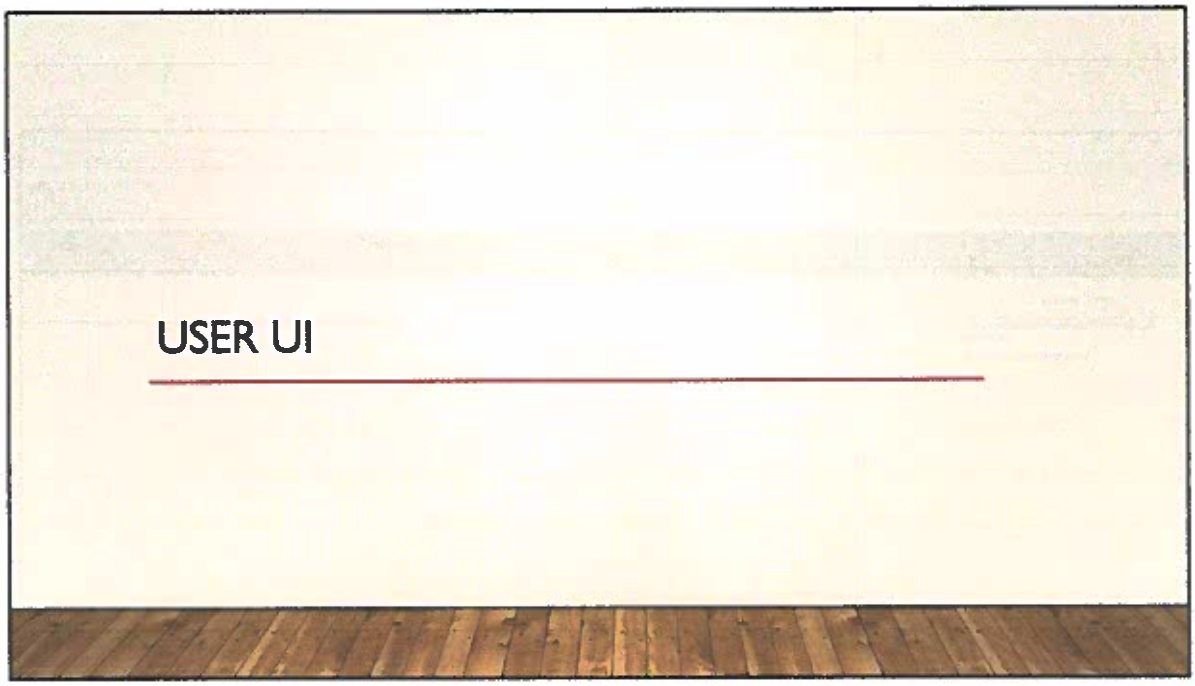
Then the proctor needs to populate the Problem directory with the problem statements, test cases, and test case solutions.



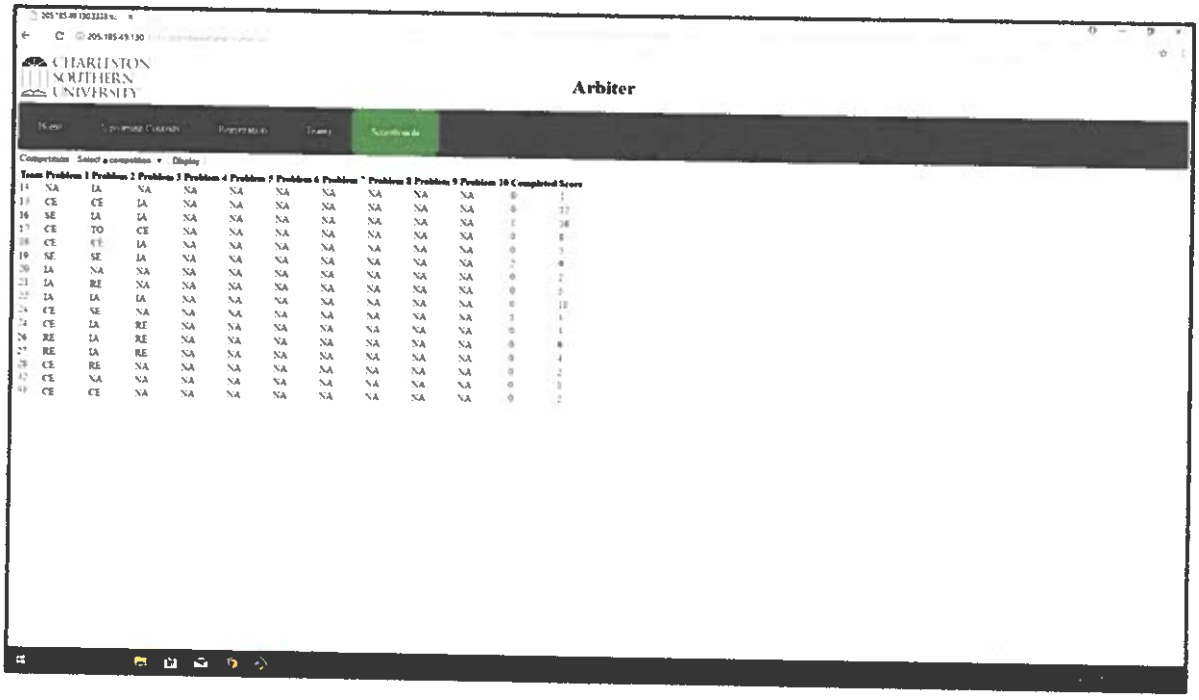
Then the proctor updates the database so the system knows his competition is using that question.



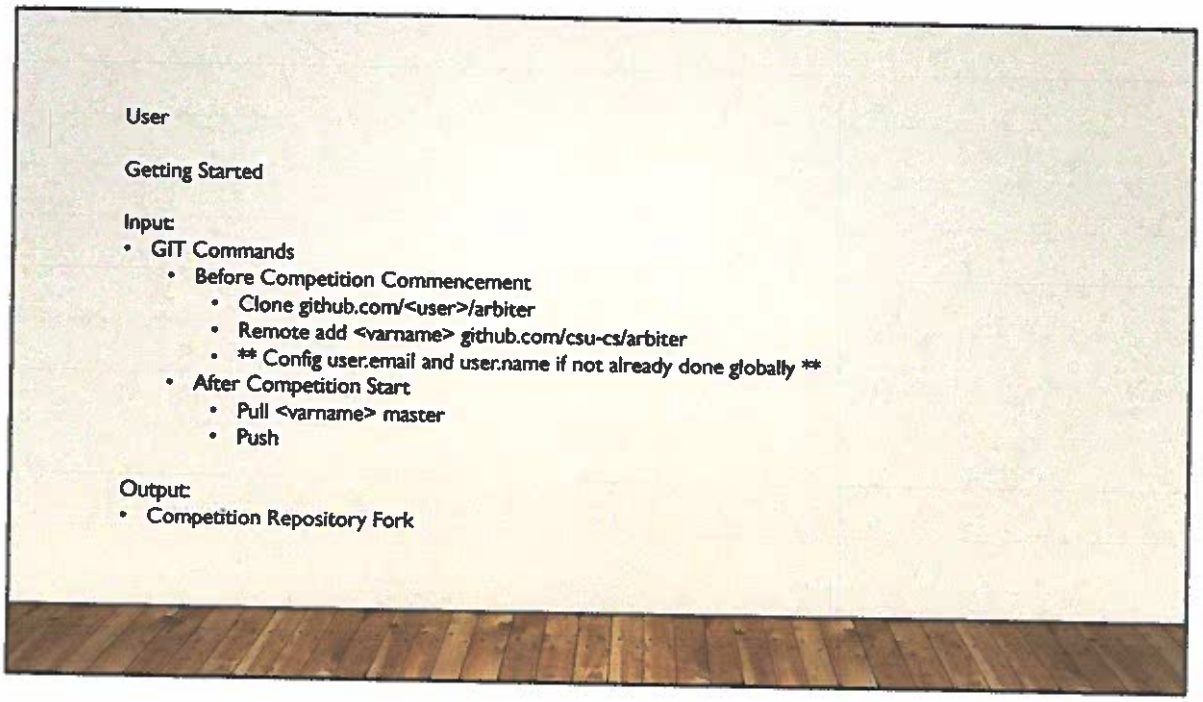
Finally the proctor runs startCompetition script which pushes the problem statements to the GITHub repo for competitors to then pull down and start competiting.



Next lets take a look at what the competitors will have to work with.



The scoreboard view, allows users to see up-to-date information on the status of their submissions.



To compete:

GIT Command Line Interface:

Users must clone their forked repository.
After start time, pull the latest update.
After source code is finished, add the file and push it

GITHub:

Users navigate to their GITHub forked repo.
After start time, refresh the page to view the problems.
Enter the problem directory and create the source code.
Commit directly from the webpage.

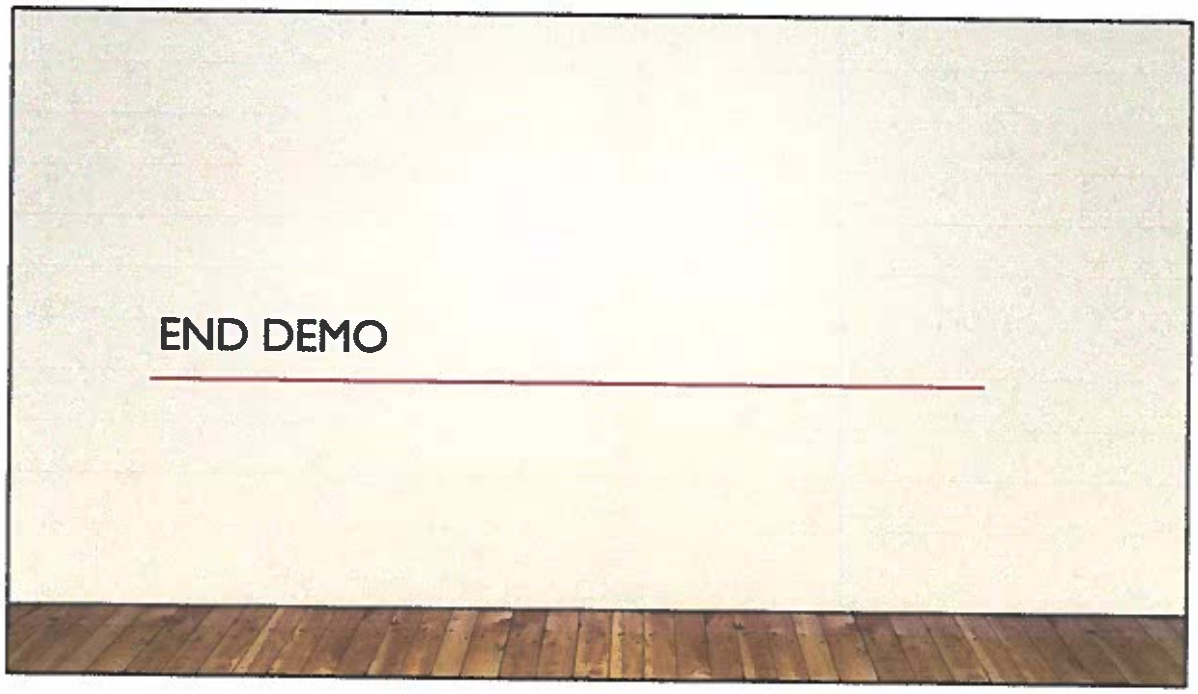
User

Submission

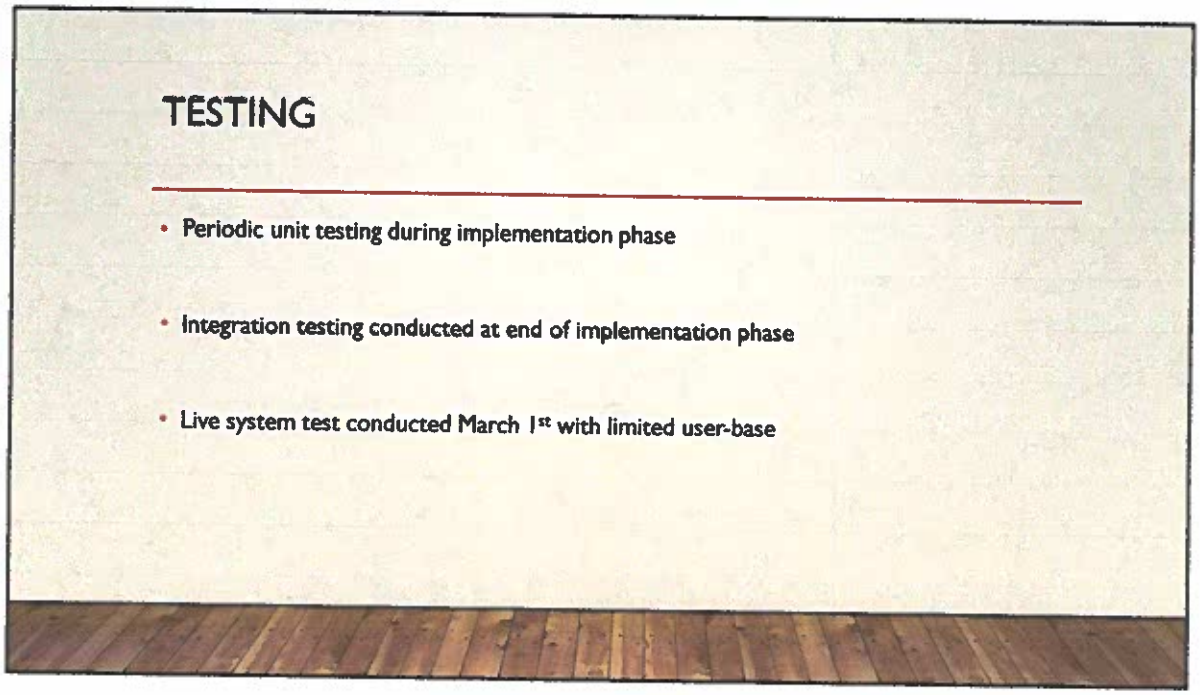
Input:
- Source Code
- GIT Commands (Push)

Output:
- Database Updates

If the user chose to use the GIT Command Line; they will find the problem directories after start time and can create source code files directly in the problem directories.



That's the process users will undergo, next lets look at how the system was developed.



Module testing was conducted after every module implementation and after every modification.

Integration testing included a mock competition administered and taken by the project creator.

System testing included a live competition administered by the project creator but taken by a select group of users who provided post-test feedback.

SURVEY METRICS FROM SYSTEM TEST

Question	Average
How would you rate Arbiter?	2.18
Satisfied with reliability?	2.45
Satisfied with security?	2.00
Satisfied with ease of use?	2.27
Satisfied with look and feel?	2.00
Satisfied with account setup?	1.45

SCALE

0 – Poor / Not at all satisfied

1 – Fair / Somewhat not satisfied

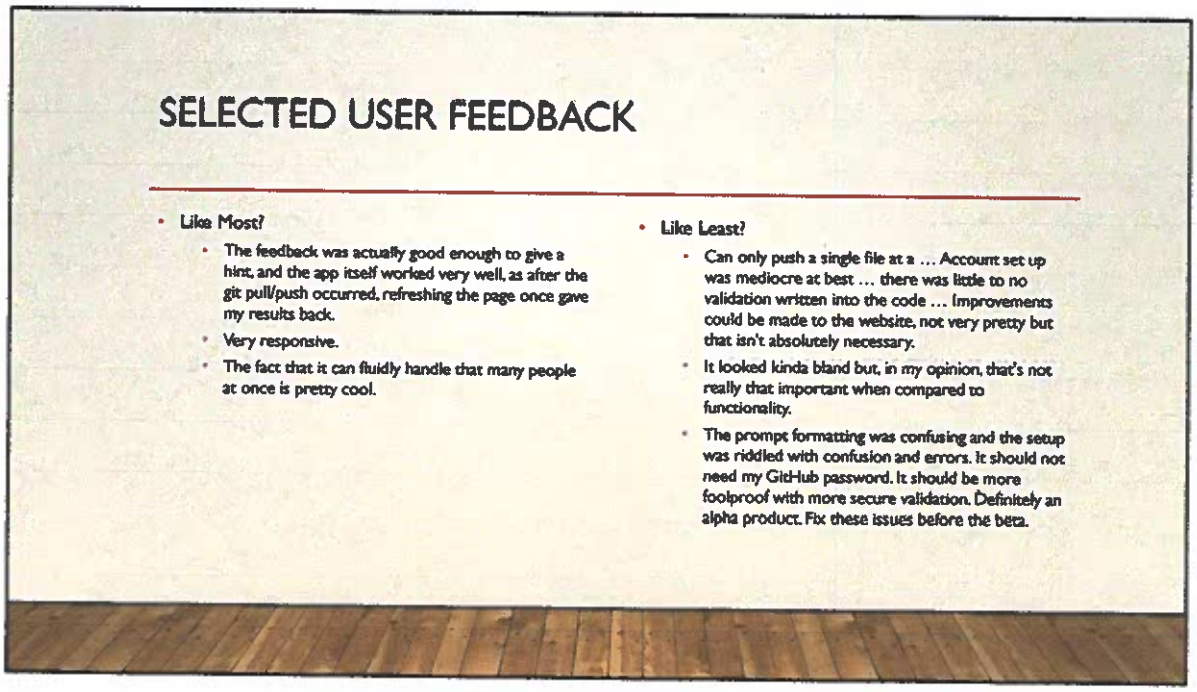
2 – Good / Somewhat satisfied

3 – Very Good / Very satisfied

4 – Excellent / Extremely satisfied

Overall users were somewhat satisfied with the system.

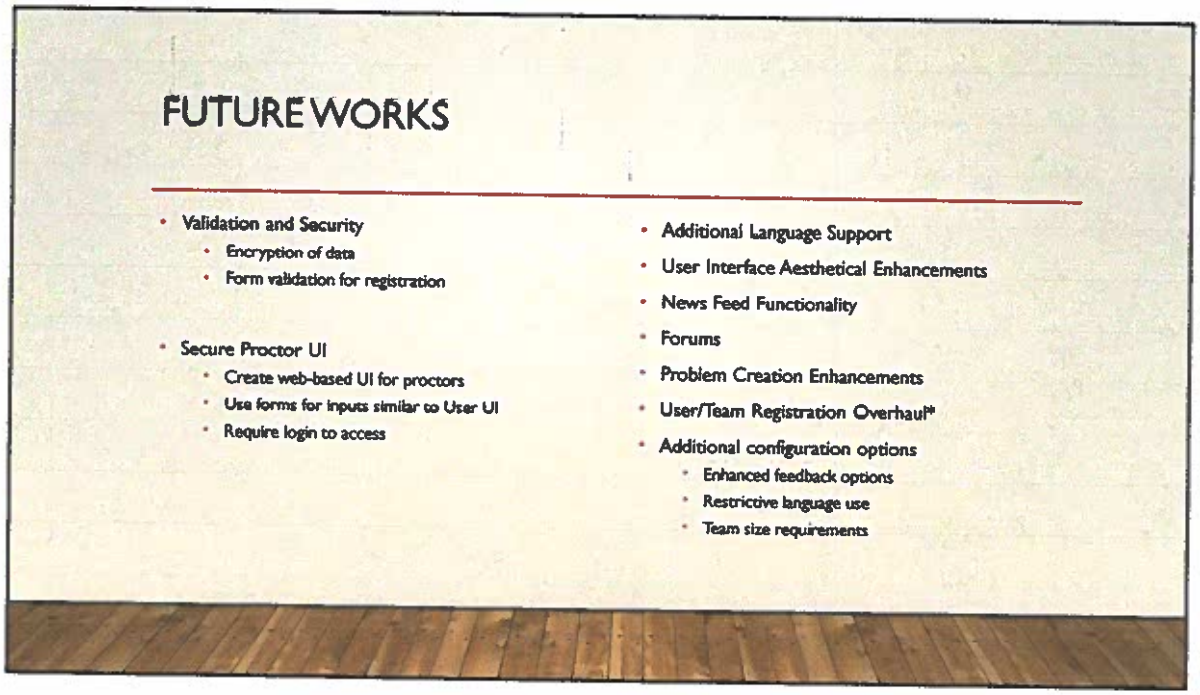
Lets look at the comments.



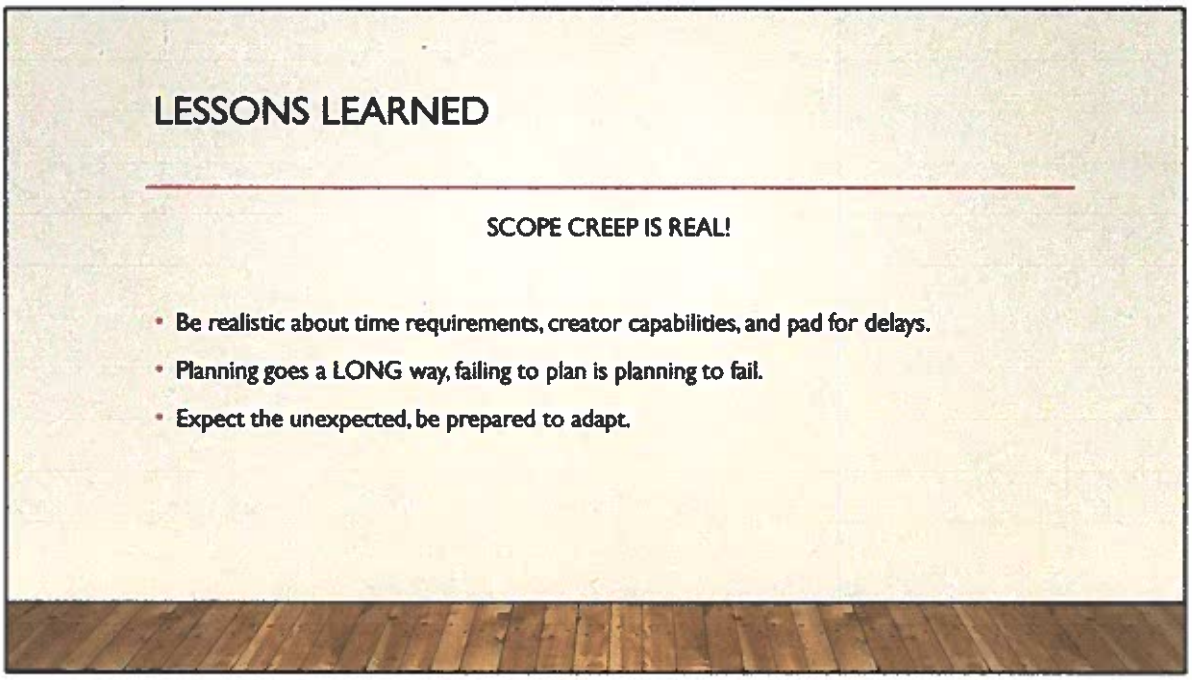
Synopsis:

Functionally great, aesthetically poor.

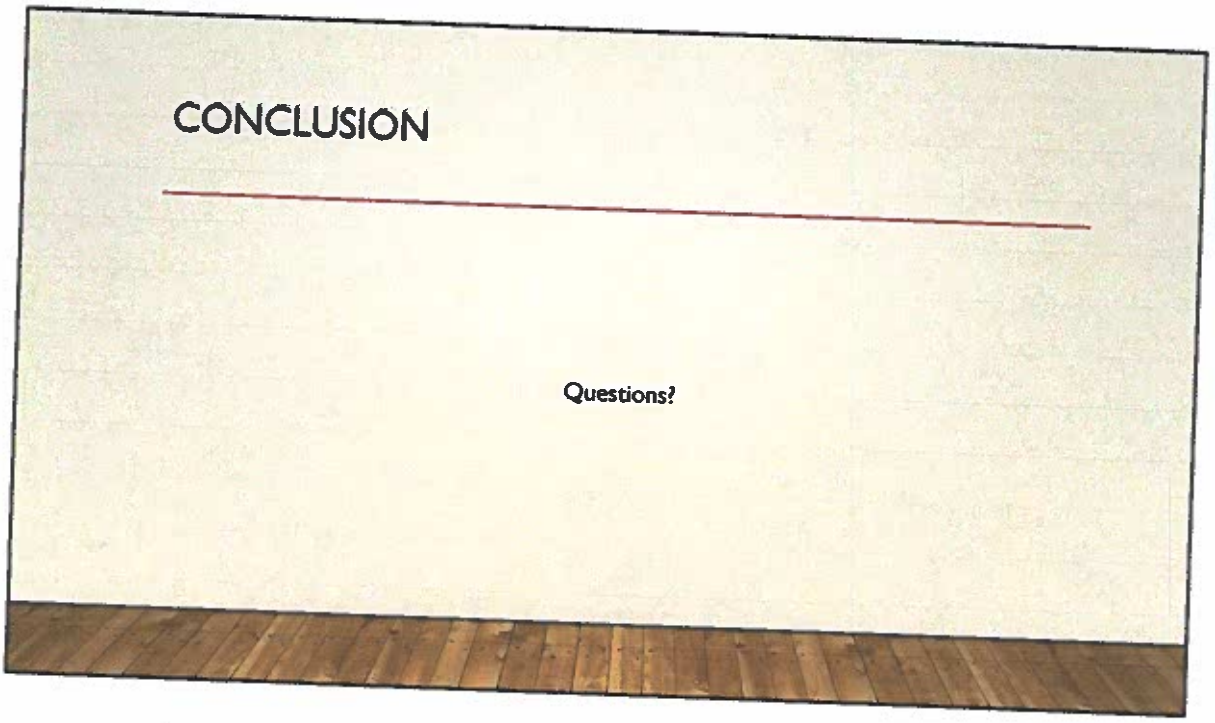
Minor bugs were discovered in the registration process and some optimizations could be made.



What the program needs... slide says it all.



In the end I found I had bitten off way more than I can handle and that I need to set realistic expectations for myself. I'm proud of what I accomplished after-the-fact but am disappointed that I didn't get as much done as I would have liked from the beginning.



Thank you for taking the time to look at my senior project!

ARBITER

A COMPETITIVE PROGRAMMING SYSTEM

DESIGNED BY: CHRISTOPHER A AUMEN, 1631!9

FOR CHARLESTON SOUTHERN UNIVERSITY

ADVISED BY: DR. PAUL WEST

PROJECT DESCRIPTION

- This project will provide a unique competitive programming system for Charleston Southern University.
- The uniqueness is based on the similarity to how a real-world application might be developed as opposed to traditional competitive systems.
- Submissions will be sent to a revision control system, GIT, that can then be pulled by the judged and reviewed.
- Additionally the increasing use of remote developers and internet based networking prompted this system to utilize the internet as the primary submission method.

LANGUAGES

- The interface for users will be webpage based using HTML and Javascript.
- The database will be using SQL.
- The primary interactions will be run in Javascript.

MAJOR GOAL DATES

- Design – Spring, 2017
  - Proposal Due Date: April 18, 2017
- Develop – Fall, 2017
  - Weekly progress goals available in Design Document
  - First Test Phase: Oct. 30 – Nov. 3, 2017
  - Initial Expected Release: Dec. 8, 2017
- Deploy – Spring, 2018
  - Live Test by Designer: February, 2018
  - Live Test by 3rd Party Proctor: March/April, 2018
  - Presentable working version: April, 2018

HARDWARE REQUIREMENTS

- Initial:
  - 1 Server with
    - Web server
    - Database server (SQL)
    - GITHub access
    - C++ compiler
    - Java compiler
  - 2 Clients with
    - Web browser (multiple preferred, one required)
    - GITHub access
- Final:
  - 1 Server with
    - SAME AS INITIAL +
    - Any language compiler desired
  - 10 Clients with
    - SAME AS INITIAL
  - 1 Client with
    - Remote access to server files

TESTING

- Weekly testing on each module will be done.
  - This should ensure each module is working as intended individually and will not depend on any other module.
- An initial multistage test will be done during the 10th week of development.
  - This will ensure that each module can interface with the others and work together appropriately.
  - Follow on tests will be done as fixes are applied.
- A live test will be done early in the Spring 2018 semester and shortly before final presentations.
  - This will ensure actual data can be processed as expected.

FINAL THOUGHTS

- After researching alternative means, the designer realizes that the proposed system will not be the most effective training system for competitive programming.
- For more effective training of a competitive programming team it is recommended that DomJudge be setup as it is used by the ACM ICPC Southeast Regional Competition and is already stable and efficient for it's intended purpose. The system will be much faster and more efficient than an internet and GIT based system.
- The proposed system however will provide the designer the chance to develop his own understanding of concepts learned while studying Computer Science and best demonstrate his ability to develop and deploy a working program.